MongoDB Backup and Restore

MongoDB built-in backup tool is "`mongodump`" and restore tool is "`mongorestore`". This toold backups the database in BSON format and allows full or partial restores. This tools needs to be installed separately by installing Database Tools package and it is not part of standard Server installation.

Overview of mongodump & mongorestore

- **mongodump**: Logical backup tool. It reads data through the MongoDB server (like a client) and exports it as **BSON** files (binary JSON).
- **mongorestore**: Imports BSON data back into a running mongod/mongos.

Key Characteristics

- Logical backups → slower than filesystem snapshots for very large DBs (> few hundred GB), but more flexible (can restore to different versions, filter data, rename namespaces).
- Works on **running** instances (no downtime required).
- Best suited for **small to medium** databases (< 100–200 GB recommended for performance reasons).
- Supports **authentication**, **TLS**, **compression**, **parallelism**, and **URI connection strings**.

Two Backup Formats

1. *Directory Format (--out)*

- Creates a directory structure: dump/<database>/<collection>.bson + metadata .json files (indexes, options, UUID, etc.).
- Easier to inspect, restore single collections, or partial restores.
- Can be compressed with --gzip (creates .bson.gz files).

Example — Full backup (directory):

Bash

```
mongodump --uri="mongodb://admin:pass12345@localhost:27017/?authSource=admin" \
  --out=/backups/full-dump-$(date +%Y%m%d_%H%M) \
  --gzip \
  --numParallelCollections=8
```

Single database:

Bash

```
mongodump --db=production --out=/backups/prod-dump --gzip
```

Single collection:

Bash

```
mongodump --db=production --collection=orders --out=/backups/orders-dump
```

2. Archive Format (--archive)

- Single binary file containing all databases/collections + metadata.
- More efficient for streaming, piping, and large backups.
- Smaller I/O overhead compared to many small files.
- Recommended for most production backups.

Example — Archive backup:

Bash

```
mongodump --uri="mongodb://admin:password@localhost:27017/?authSource=admin" \
  --archive=/backups/full-backup-$(date +%Y%m%d_%H%M).archive.gz \
  --gzip \
  --numParallelCollections=8
```

Compressed archive (highly recommended):

Bash

```
mongodump --archive=/backups/full-$(date +%F).archive --gzip
```

Pipe to compression (alternative):

Bash

```
mongodump --archive=- | gzip > /backups/full.archive.gz
```

Restore from archive:

Bash

```
mongorestore --archive=/backups/full-backup.archive.gz --gzip
```

mongodump & mongorestore on Standalone Instance

Backup (Standalone):

Bash

```
# Directory format
mongodump --host=localhost --port=27017 -u admin -p password --
authenticationDatabase=admin \
  --out=/backups/standalone-dump-$(date +%Y%m%d)


# Archive format (preferred)
mongodump --host=localhost --port=27017 -u admin -p password --
authenticationDatabase=admin \
  --archive=/backups/standalone-$(date +%Y%m%d_%H%M).archive --gzip
```

Restore to Standalone:

Bash

```
# From directory
mongorestore --host=localhost --port=27017 -u admin -p password --
authenticationDatabase=admin \
  /backups/standalone-dump-20260430
```

```
# From archive
mongorestore --archive=/backups/standalone-20260430.archive --gzip \
  --drop   # drops existing collections before restore
```

Useful flags:

- --drop: Drop collections before inserting (clean restore).
- --nsInclude=dbname.* or --nsExclude: Namespace filtering.
- --numInsertionWorkersPerCollection=16: Increase parallelism during restore.
- --stopOnError: Stop on first error.

mongodump & mongorestore on Self-Managed Replica Set

Best Practice for Replica Sets:

- Always run mongodump from a **secondary** member to reduce impact on primary.
- Use --oplog for **consistent** backups (highly recommended).

Backup from Secondary with --oplog (Directory):

Bash

```
mongodump --host=secondary-host:27017 -u admin -p password --
authenticationDatabase=admin \
  --out=/backups/rs-dump-$(date +%Y%m%d) \
  --oplog \
  --gzip
```

Backup with --oplog (Archive):

Bash

```
mongodump --host=secondary-host:27017 \
  --archive=/backups/rs-full-$(date +%Y%m%d).archive --gzip \
  --oplog
```

Note: --oplog works **only** on a full instance dump (no --db or --collection restriction). It creates an oplog.bson file at the root containing operations that occurred *during* the dump.

Restoring a Single Collection from a Full Backup

From Directory Format:

Bash

```
# Restore only one collection
mongorestore --db=production --collection=orders \
  /backups/full-dump-20260430/production/orders.bson \
  --drop
```

From Archive Format (use namespace filtering):

Bash

```
mongorestore --archive=/backups/full-backup.archive --gzip \
  --nsInclude=production.orders \
  --drop
```

Rename during restore:

Bash

```
mongorestore --archive=full.archive --gzip \
  --nsFrom=production.orders \
  --nsTo=production.orders_restored
```

Oplog, --oplog, and --oplogReplay

What is Oplog?

The **oplog** (local.oplog.rs) is a capped collection on every replica set member that records **all write operations** in timestamp order. It enables replication and **Point-in-Time Recovery (PITR)**.

When to Use --oplog (in mongodump)

- When a ReplicaSet id backedup
- To get a **consistent point-in-time** view of the entire instance.
- Captures changes that happen *while* the dump is running.
- **Required** if you want to do PITR.

Command:

Bash

```
mongodump ... --oplog
This adds oplog.bson to the dump (or inside the archive).
mongorestore --oplogReplay
```

- Applies the oplog.bson after restoring data.
- Brings the restored database to the exact state at the **end** of the dump.

Full restore with oplog replay:

Bash

```
mongorestore --archive=rs-full.archive --gzip --oplogReplay --drop
```

Point-in-Time Recovery (PITR) using Oplog

Classic PITR workflow (self-managed):

First Restore from backup

```
mongorestore --archive=base.archive --gzip --oplogReplay
```

Replay additional oplog up to a specific timestamp:

Bash

```
mongorestore --oplogReplay --oplogLimit="Timestamp(1745976000, 0)" oplog-dump-dir/
```

Format of --oplogLimit: seconds:increment (Unix timestamp).

How to find a good timestamp?

Bash

 # On a replica set member

```
mongosh --eval 'db.getSiblingDB("local").oplog.rs.find({}, {ts:1}).sort({ts:-1}).limit(5)'
```

Oplog Relay Concept:

- Some advanced setups (or third-party tools like Percona Backup for MongoDB) continuously "relay" or tail the oplog and store it separately.
- You can manually dump portions of oplog.rs with a query on ts and replay them later using --oplogReplay --oplogLimit.

Important Limitations:

- --oplogReplay works **only** on **full instance** restores (not single DB/collection).
- You cannot mix --oplogReplay with partial restores easily.

Best Practices for Self-Deployed Environments

- Prefer **archive + gzip** for most cases.
- Run backups from **secondaries**.
- Use --oplog for replica sets.
- Increase parallelism: --numParallelCollections=8 (dump), --numInsertionWorkersPerCollection=16 (restore).
- Monitor impact: Use --readPreference=secondary explicitly.
- Encrypt backups at rest or in transit.
- Test restores regularly.
- For very large datasets (>200–300 GB), consider filesystem snapshots + mongod --repair or Percona Backup for MongoDB (PBM) instead of pure mongodump.
- In MongoDB 8.0: Faster query execution and storage improvements make logical backups slightly more efficient, but the tool behavior remains the same.

Script to manually dump the OpLog

Dump last 1 hour of oplog

```
mongodump \
  --uri="mongodb://admin:password@localhost:27017/?authSource=admin" \
  --db=local \
  --collection=oplog.rs \
  --query='{ "ts": { "$gte": { "$timestamp": { "t": '$(($(date +%s)-3600))', "i": 0 } } } }' \
  --archive=/backups/oplog/oplog-last24h-$(date +%Y%m%d_%H%M).archive -gzip
```

BSON Dump

Converts .bson files (produced by mongodump) into readable JSON.

Syntax

Bash

```
bsondump [options] <bsonfile>
```

Options

Option	Description
--outFile	Output file (default: stdout)
--pretty	Pretty print JSON
--type	json (default) or debug
--bsonType	Specify if input is not standard collection BSON

Examples

Basic usage

Bash

```
bsondump /backups/full-dump/mydb/orders.bson > orders.json
```

Pretty print

Bash

```
bsondump --pretty /backups/full-dump/mydb/users.bson > users_pretty.json
```

Convert specific collection from archive backup

Bash

 # First extract bson from archive (if needed), then:

```
bsondump --pretty dump/mydb/sessions.bson
```

When to use bsondump

- Debugging what is actually inside a .bson file from mongodump
- Inspecting backup content without restoring
- Converting old backups to JSON for analysis
- Troubleshooting data issues

Exporting And Importing Data

mongoexport

Exports data from a collection into JSON or CSV format.

Complete Syntax

Bash

```
mongoexport [options] [--collection <coll>] [--out <file>]
```

Most Important Options

Option	Description
`--uri`	Connection string (recommended)
`--host, --port`	Host and port
`-u, --username`	Username
`-p, --password`	Password
`--authenticationDatabase`	Usually admin
`--db, -d`	Database name
`--collection, -c`	Collection name
`--out, -o`	Output file
`--type`	json (default), csv, tsv
`--fields`	Comma-separated fields for CSV
`--query, -q`	Filter documents (JSON query)
`--sort`	Sort order
`--limit`	Limit number of documents
`--skip`	Skip documents
`--jsonArray`	Output as single JSON array (instead of NDJSON)
`--pretty`	Pretty-print JSON
`--gzip`	Compress output with gzip
`--readPreference`	secondary, secondaryPreferred (recommended for large exports)

Examples

Export entire collection as JSON (NDJSON - Newline Delimited)

Bash

```
mongoexport --uri="mongodb://admin:password@localhost:27017/mydb?authSource=admin" \
  --collection=users \
  --out=/exports/users.json \
  --pretty
```

Export as JSON Array

Bash

```
mongoexport --uri="mongodb://admin:password@localhost:27017/mydb?authSource=admin" \
  --collection=orders \
  --out=/exports/orders_array.json \
  --jsonArray
```

Export specific fields as CSV

Bash

```
mongoexport --uri="mongodb://admin:password@localhost:27017/mydb?authSource=admin" \
  --collection=products \
  --type=csv \
  --fields=_id,name,price,category,status \
  --out=/exports/products.csv
```

Export with query filter (last 7 days)

Bash

```
mongoexport --db=mydb --collection=logs \
  --query='{"createdAt": {"$gte": {"$date": "2026-04-23T00:00:00Z"}}}' \
  --out=/exports/recent_logs.json \
  --gzip
```

Export from Replica Set

Bash

```
mongoexport --
uri="mongodb://admin:password@rs1:27017,rs2:27017,rs3:27017/mydb?authSource=admin&repl
icaSet=myrs&readPreference=secondary" \
  --collection=analytics \
  --out=/exports/analytics.json
```

mongoimport

Imports data from JSON, CSV, or TSV files into MongoDB.

Complete Syntax

Bash

```
mongoimport [options] [--file <file>]
```

Important Options

Option	Description
`--db, -d`	Target database
`--collection, -c`	Target collection
`--file`	Input file
`--type`	json, csv, tsv

Option	Description
`--mode`	`insert (default), upsert, merge`
`--upsertFields`	`Fields to match for upsert/merge`
`--drop`	`Drop collection before import`
`--headerline`	`First row is header (for CSV)`
`--ignoreBlanks`	`Ignore blank fields in CSV`
`--numInsertionWorkers`	`Parallel workers (default 1, increase for speed)`
`--jsonArray`	`Input is a JSON array`

Examples

Import JSON (NDJSON)

Bash

```
mongoimport --uri="mongodb://admin:password@localhost:27017/mydb?authSource=admin" \
  --collection=users \
  --file=/exports/users.json \
  --numInsertionWorkers=8
```

Import JSON Array

Bash

```
mongoimport --db=mydb --collection=orders \
  --file=orders_array.json \
  --jsonArray \
  --drop
```

Import CSV with header

Bash

```
mongoimport --db=mydb --collection=products \
  --type=csv \
  --file=products.csv \
  --headerline \
  --fields=_id,name,price,category \
  --numInsertionWorkers=16
```

Upsert using unique field

Bash

```
mongoimport --db=mydb --collection=customers \
  --file=customers.json \
  --mode=upsert \
```

```
--upsertFields=email \
--numInsertionWorkers=8
```